

北方工业大学

2025-2026 学年第二学期课设报告



题目： 开源鸿蒙赋能智慧农业

——基于 Hi3861 与 HarmonyOS 的智慧大棚控制终端

课程名称：鸿蒙项目制综合实践

组别：清霁

组长：陈梓睿

软件名称：Smart Shed

指导教师：王岚

2026 年 6 月 6 日

North China University of Technology

2025-2026 First Semester

Course Design Report



Title:

OpenHarmony Empowers Smart Agriculture
—— Smart Greenhouse Control Terminal Based on
Hi3861 and HarmonyOS

Course Name: HarmonyOS Full-Stack Project-Based Practice

Group: Qingji

Leader: Ray Chen (Rayawa)

Software Name: Smart Shed

Teacher: Wang Lan

2026/6/6

论文标题

[illegible][illegible]

关键词: 关键词 1; 关键词 2; 关键词 3; 关键词 4

Title

[illegible]

Second paragraph text text text text text text text text text text text, text text
text text text text text, text text text text text text text text text text text text. Text text text
text text text text text text text text text text text text, text text text text text. Second paragraph
text text text text text text text text text text text text text text text, text text text text text text text, text
text text text text text text text text text text text text text text. Text text text text text text text text text
text text text text text text, text text text text text.

Keywords: keywords1; keywords2; keywords3; keywords4

目录

一、绪论

1. 项目背景

传统农业大棚的温湿度控制、光照调节、土壤浇灌等工作高度依赖人力巡检与人工操作，存在效率低、反应滞后、精确度差等问题。随着物联网（IoT）技术的发展和国产芯片/操作系统的崛起，利用开源鸿蒙生态构建低成本、高可靠的智慧农业控制系统，不仅具有现实的应用价值，也契合国家信创产业的发展方向。

本课程设计以 Hi3861 为底层感知与执行平台，搭载开源鸿蒙 OpenHarmony LiteOS-M 实时操作系统；以 HarmonyOS 平板/手机为上层控制终端，采用 ArkTS + ArkUI 构建现代移动端应用；通过 MQTT 协议实现端云协同，打造一套完整的智慧大棚环境监控与自动控制系统——Smart Shed。

2. 设计目标

本项目的设计目标主要有以下几个方面：

- (1) **实时环境监测**：系统需能够实时采集大棚内的关键环境参数，包括空气温湿度、光照强度及土壤湿度，为后续控制决策提供数据基础；
- (2) **远程手动控制**：用户可通过移动端应用程序，对补光灯、风扇、水泵等执行器进行远程手动操控，实现对温室设备的灵活干预；
- (3) **智能自动调节**：系统应具备阈值设定与自动决策能力。在用户预设环境参数阈值后，系统可根据实时采集数据自动触发相应执行器，无需人工介入；
- (4) **稳定可靠通信**：基于 MQTT 协议构建双向实时通信机制，支持断线重连与心跳保活功能，确保控制指令与环境数据传输的稳定性与可靠性；
- (5) **良好用户体验**：移动端界面遵循 HDS 设计系统，支持深色与浅色模式的自适应切换，适配平板设备的分栏布局，并融入沉浸式动效，提升用户操作体验。

3. 系统架构

本系统采用三层架构，分别为嵌入式端（设备层）、服务端（通信层）与应用端（用户层），各层具体选型与配置如下：

(1) MQTTX 服务端

服务端采用 MQTT 5.0 协议，并通过 WebSocket 进行通信接入。具体连接地址为 wss://broker.emqx.io:8084，客户端工具选用 MQTTX 进行调试与连接验证。该方案支持双向异步通信、心跳保活及断线重连机制，满足系统对稳定可靠通信的需求。

(2) Hi3861 嵌入式端

嵌入式端基于开源操作系统 OpenHarmony 1.0 Release 进行开发。核心控制单元选用海思 Hi3861 芯片方案，分为 rgb 与 soi 两块开发板实现所有硬件功能；

(3) HarmonyOS 应用端

应用端基于 HarmonyOS 6.1.0 Release (API 23) 进行开发，面向所有华为设备进行响应式适配。应用名称 Smart Shed，采用 HDS 设计系统，支持深色/浅色模式自适应、平板分栏布局及沉浸式动效并使用新时代页面路由栈，符合面向未来的标准鸿蒙设计语言，实现良好的跨设备用户体验。

4. 开发环境

本系统开发环境分为嵌入式端与应用端两部分，主要软硬件平台、操作系统、工具链及通信组件配置如下

组件	版本 / 规格
开发板	HiSilicon Hi3861 (RISC-V, 160KB SRAM, 2MB Flash)
嵌入式 OS	OpenHarmony 1.0 Release / LiteOS-M
嵌入式工具	DevEco Device Tool, GN+Ninja, hb, HiBurn
编译环境	Ubuntu 24.04 LTS Server (aarch64)
MQTT 库(嵌入式)	Eclipse Paho MQTTPacket (C)
TCP/IP 栈	lwIP
应用端 OS	HarmonyOS 6.1.0 Release (API 23)
使用 IDE	Visual Studio Code, DevEco Studio 6.1.0 Release
应用端变成语言	ArkTS
UI 框架	ArkUI + HDS Design System
网络 Kit	@kit.NetworkKit (socket.TCPSocket)
MQTT Broker	broker.emqx.io:1883 (EMQX)

5. 技术栈

本系统技术栈分为服务端、嵌入式端与应用端三部分。

(1) **服务端**基于 MQTT 5.0 协议，通过 WebSocket 客户端运行于浏览器，接入地址为 Rayawa 提供的 wss://broker.emqx.io:8084。

(2) **嵌入式端**以 HiSilicon Hi3861 芯片为核心，运行 OpenHarmony 1.0 Release 及 LiteOS 内核，开发语言主要为 C 与 Python 3.8，采用 paho MQTT 实现 MQTT 通信，编译与烧录工具链包括 GN、hb、HiBurn 及 IPOP，整体编译环境部署于 Ubuntu 24.04 LTS Server (aarch64)。

(3) **应用端**采用 HarmonyOS 6.1.0 Release (API 23)，开发环境为 DevEco Studio 6.1.0 Release，开发语言使用 ArkTS，UI 框架基于 ArkUI，构建工具为 Hvigor。

二、MQTTX 服务端设计

1. 发布/订阅架构

本系统采用经典的发布/订阅架构，包含发布者 (Publisher)、代理服务器 (Broker) 与订阅者 (Subscriber) 三部分。

(1) **发布者**分为两类：其一为 Hi3861 嵌入式端，通过 I2C 与 ADC 接口定时采集温湿度、光照、气压等环境参数，并利用 Paho MQTT 库将数据封装为 JSON 格式，发布至指定主题。其二为应用客户端，根据用户在移动端 UI 上的操作生成控制报文（如风扇挡位、继电器状态），实时推送至云端控制主题。

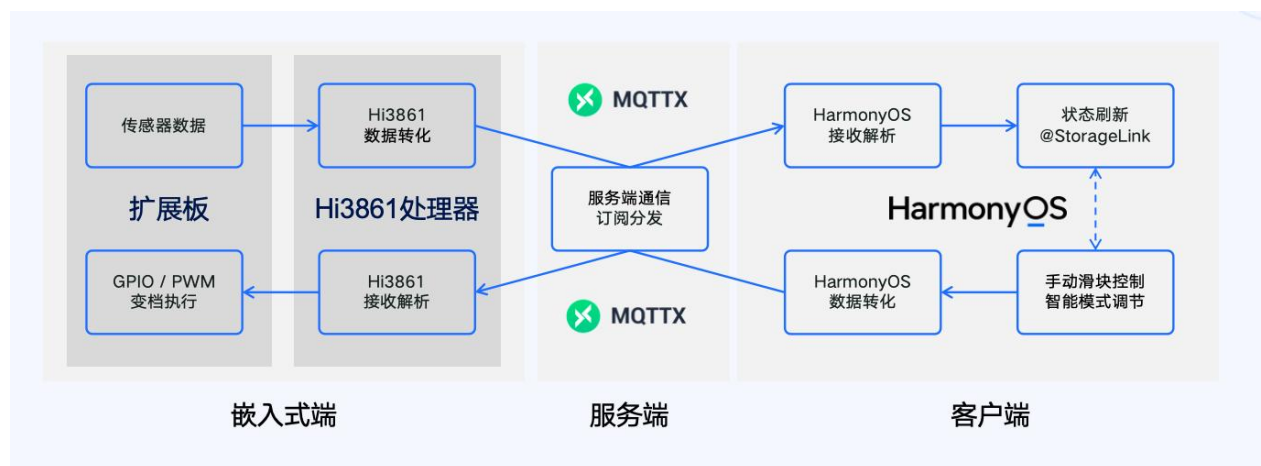
(2) **代理服务器**采用 EMQX 公网分布式集群作为云端 Broker，基于 Erlang/OTP 平台实现高并发连接管理，不存储持久数据。其内部路由引擎通过通配符匹配与过滤分发策略，在毫秒级内将消息精准投递至订阅端，并处理 QoS 握手校验，确保网络波动下消息的可靠

到达。

(3) 订阅亦分为两类: 应用客户端订阅环境状态主题, 收到 Broker 转发的传感器 JSON 数据后, 通过回调函数解析并实时展示在 UI 组件上。Hi3861 订阅控制主题, 在循环中调用 MQTTPacket_read 监听报文, 收到控制指令后解析 Payload, 通过 GPIO 或 PWM 接口控制外设硬件。

2. 数据流展示

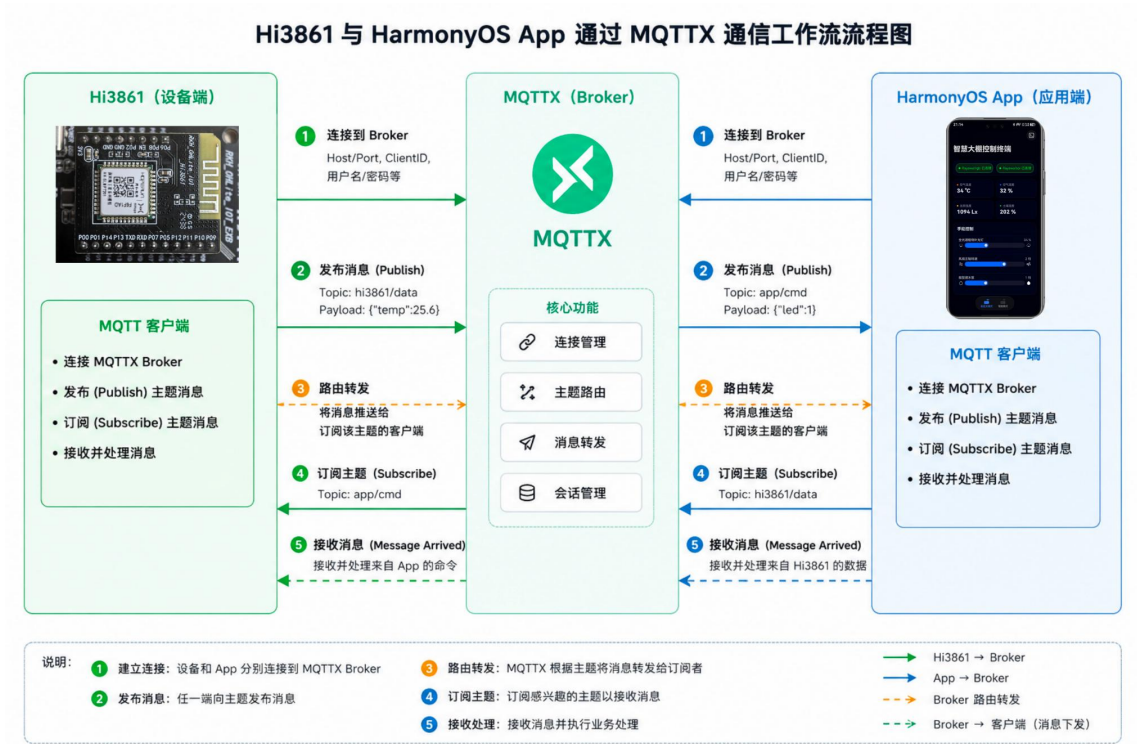
本系统采用 MQTT 协议作为轻量级数据传输载体, 通过 MQTTX 代理 (Broker) 实现了 Hi3861 硬件设备端与 HarmonyOS 移动应用端的高效双向异步通信。具体通信流转过程如下:



(1) **连接建立阶段:** 系统初始化后, Hi3861 设备端与 HarmonyOS App 分别配置对应的 Host 地址、端口号、客户端 ID (ClientID) 及鉴权信息, 向 MQTTX Broker 发起连接请求, 建立稳定的 TCP 长连接。

(2) **上行数据流 (设备监测):** Hi3861 设备端通过“MQTT 客户端”模块, 将采集到的环境或传感器数据封装为 JSON 格式的 Payload (例如 {"temp": 25.6}), 并指定主题 hi3861/data 向上发布 (Publish)。同时, HarmonyOS App 提前订阅 (Subscribe) 了该主题。MQTTX Broker 触发主题路由与消息转发机制, 将该数据实时推送至 App 端, 应用端接收解析后更新用户界面。

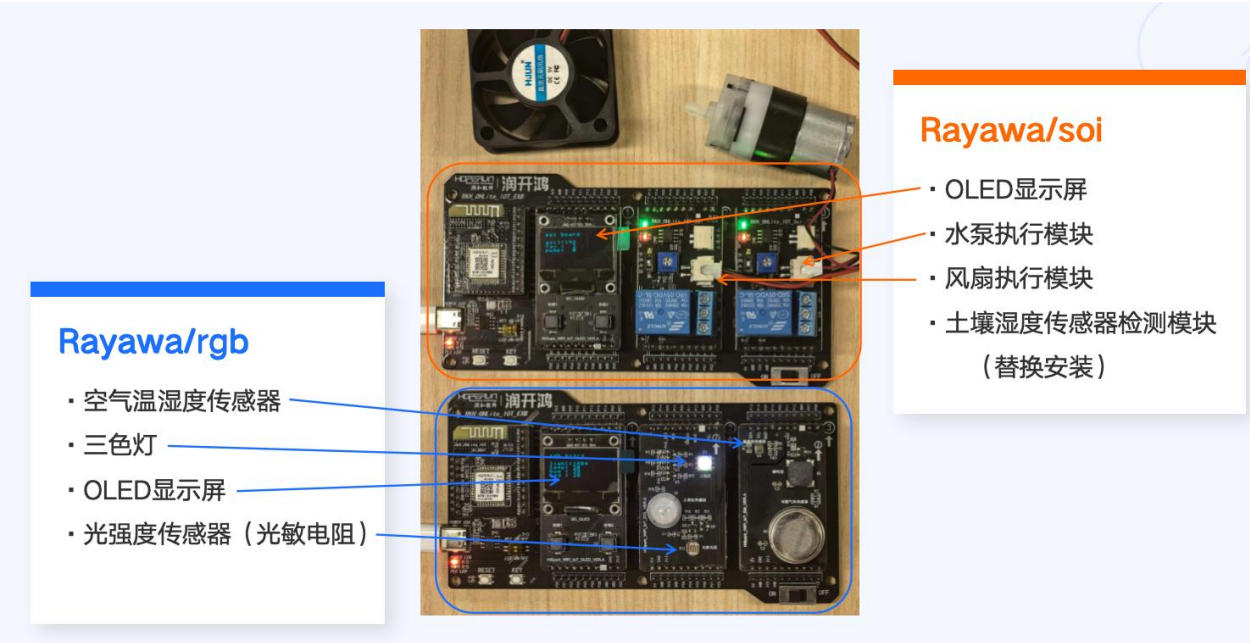
(3) **下行控制流 (应用反控):** 当用户在 HarmonyOS App 上进行控制操作时, App 端向主题 app/cmd 发布控制指令 (例如 {"led": 1})。由于 Hi3861 设备端已处于该主题的订阅状态, MQTTX Broker 将指令准确转发至硬件端, 硬件端接收解析 (Message Arrived) 后立即执行相应的硬件外设驱动逻辑。



三、Hi3861 嵌入式端设计

1. 总览

Hi3861 嵌入式端通过扩展板将各元器件并联在开发板上。为避免引脚冲突，系统设计了两个可独立运行的模块板卡，分别为 Rayawa/rgb 与 Rayawa/soi。每个单板均能独立运行，板载元器件与实际位置的对应关系如下图所示：



各元件功能、对应接口与控制程序如下表所示：

元件	功能	硬件接口	控制程序
OLED	状态打印	I2C0-0x78 GPIO13&14	oled_ssd1306.c
空气温湿度传感器	检测空气参数	I2C0-0x44	temp_and_hum_task.c
光敏电阻	检测光照强度	ADC_CHANNEL_4	light_intensity_task.c
三色灯(RGB)	大棚光照控制	GPIO10/11/12 => PWM1/2/3	led_task.c

Rayawa/rgb

元件	功能	硬件接口	控制程序
OLED	状态打印	I2C0-0x78 GPIO13&14	oled_ssd1306.c
土壤湿度传感器	外接土壤探头	ADC_CHANNEL_4	soil_moisture_task.c
水泵	外接水泵	(P06)	water_pump_task.c
风扇	外接风扇	(P08)	fan_task.c

Rayawa/soi

2. 项目结构

嵌入式端软件项目自底向上划分为三层: 公共通用层 (common)、业务模块层 (modules) 与开发板构建层 (boards), 具体设计如图所示:



(1) 公共通用层负责统一管理 Wi-Fi 连接、MQTT 通信及 OLED 驱动, 并通过共享资源 I2C 锁机制确保多模块并发访问时的系统运行效率与稳定性。

(2) 业务模块层分为传感器与执行器两个子模块。传感器目录存放温湿度传感器、光强传感器及土壤湿度传感器的头文件与程序源文件; 执行器目录则控制风扇、补光灯和潜水泵等设备。

(3) 开发板构建层包含两个开发板的 GN 构建配置的统一管理。该层设计尤为关键——它决定了系统在面对硬件平台迭代或扩展时的适配成本。统一的 GN 管理使得新增开发板仅需补充对应构建配置, 无需改动上层业务逻辑, 从而显著提升系统的可扩展性与维护效率。

3. 程序架构设计

(1) 总线程设计

系统启动后, 通过 SYS_RUN 入口函数创建主线程, 入口文件为 smart_shed_all.c, 栈大小为 8KB。主线程按顺序初始化并拉起各功能子线程, 包括传感器采集、执行器控制、MQTT 通信及 OLED 显示。同时, 采用条件编译机制, 根据板卡配置动态启用对应模块, 实现统一的任务调度入口。

(2) 分线程设计

各功能模块独立运行于 LiteOS 实时内核中, 具体包括: 空气温湿度采集 (AHT20, I2C 接口)、光照检测、土壤湿度读取、风扇/LED/水泵的 GPIO 控制、OLED 显示刷新以及

MQTT 网络通信线程。每个线程拥有独立的栈空间（4KB 至 8KB）及优先级，并行处理互不阻塞。

(3) I2C 互斥锁设计

部分元件共用扩展板并联引脚，例如 AHT20 温湿度传感器与 OLED 显示屏共用 I2C0 总线（GPIO13 及 GPIO14）。为此，系统设计了 i2c0_lock/unlock 互斥锁机制，确保同一时刻仅有一个设备占用总线。每次读写操作前后均进行加锁与解锁操作，避免多线程并发访问导致的数据冲突与通信异常。

4. WiFi 与 MQTT 通信

设备连接 SSID 为“Rayawa”的无线网络，通过 DHCP 动态获取 IP 地址后，进入 lwIP 协议栈的 TCP 通信阶段。本方案中，Paho 库仅负责 MQTT 报文的编解码工作，TCP Socket 的完整生命周期（包括创建、连接、收发、关闭及断线重连）由开发者自行管理。

```
int packet_type = MQTTPacket_read(buf, sizeof(buf), transport_getdata);
if (packet_type == PUBLISH) {
    unsigned char dup = 0;
    int qos = 0;
    unsigned char retained = 0;
    unsigned short msgid = 0;
    int payload_len = 0;
    unsigned char *payload_in = NULL;
    MQTTString received_topic = MQTTString_initializer;
    char topic[32];

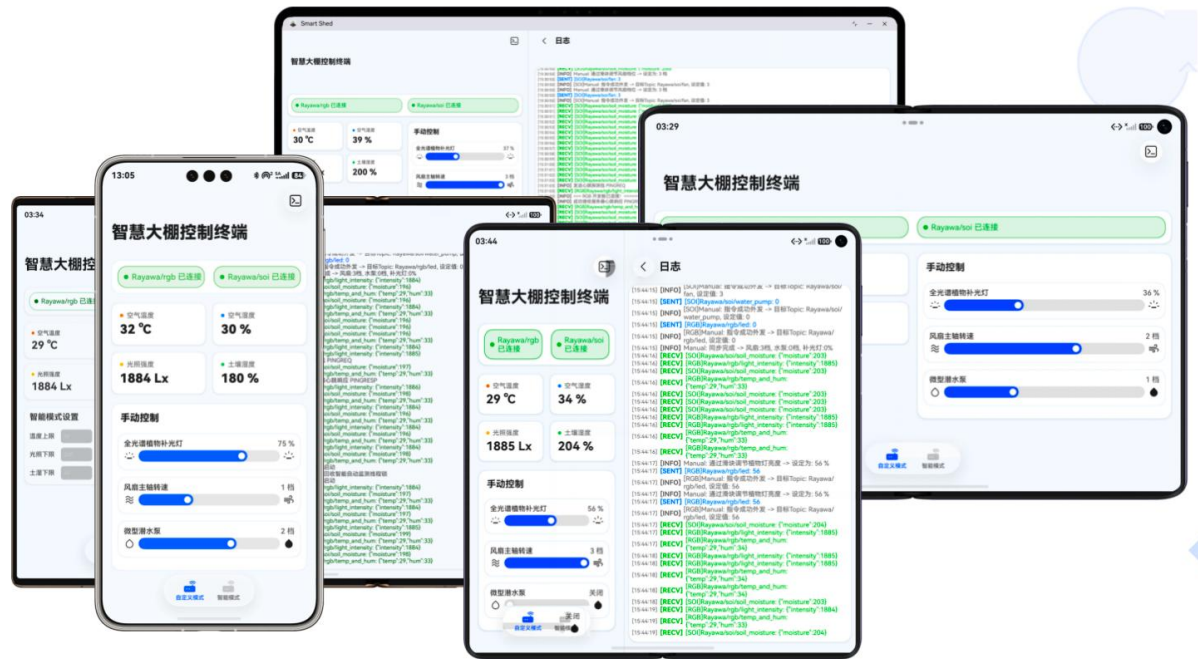
    if (MQTTDeserialize_publish(&dup, &qos, &retained, &msgid, &received_topic,
                                &payload_in, &payload_len, buf, sizeof(buf)) == 1 &&
        payload_len > 0 && payload_len < (int)sizeof(payload)) {
        int topic_len = received_topic.lenstring.len;
        if (topic_len >= (int)sizeof(topic)) {
            topic_len = sizeof(topic) - 1;
        }
        memcpy(topic, received_topic.lenstring.data, topic_len);
        topic[topic_len] = '\0';
        memcpy(payload, payload_in, payload_len);
        payload[payload_len] = '\0';
        printf("MQTT cmd topic=%s payload=%s\n", topic, payload);
        mqtt_apply_command(topic, payload);
    }
}
```

传感器数据以 JSON 格式封装后上报至云端。当设备收到 PUBLISH 报文时，系统调用 MQTTDeserialize_publish() 函数解析出主题与负载内容，随后通过 mqtt_apply_command() 函数根据主题匹配规则设置相应的全局变量。独立的执行器线程持续轮询读取这些全局变量，并根据变量状态控制 GPIO 或 PWM 输出，进而驱动大棚中的风扇、补光灯、水泵等各类执行设备，最终实现智慧大棚的自动化控制。

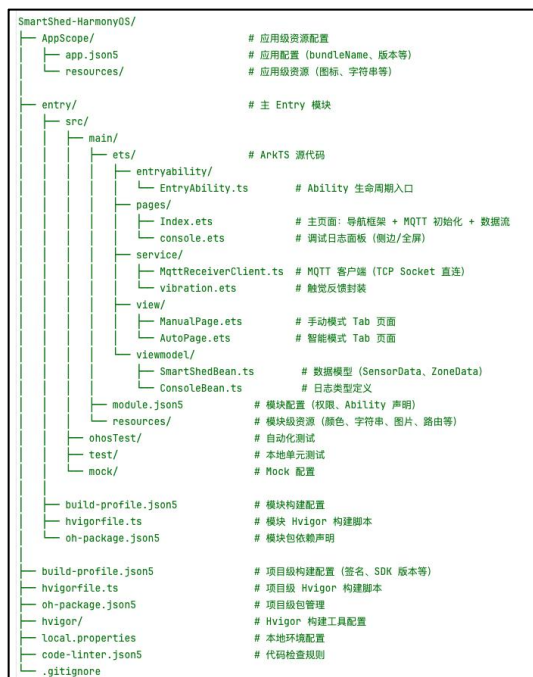
四、HarmonyOS 客户端设计

1. 总览





2. 项目结构



鸿蒙应用端采用清晰的分层架构，主要包含以下四个核心目录。

AppScope 目录负责统一管理应用级全局资源与 Hvmgr 构建脚本，为整个应用提供基础的配置与环境支持。

view 与 pages 目录基于 ArkTS 语言，构建了包含手动控制与智能自动双模式、调试面板在内的流畅用户界面。

viewmodel 目录用于定义核心数据结构，驱动应用状态的安全传递与界面间的数据同步。

service 目录封装了 MQTT 客户端，通过 TCP Socket 直连方式实现与云端的数据实时双向通信。。

3. MQTTX 客户端逻辑

(1) 申报机制

申报机制包括订阅与发布两个环节。订阅时发送 0x20 报文，调用 subscribeToTopic 函数并拼装 0x82 响应；发布时通过控制报文 0x30 调用 publish() 函数，底层经由 TCP Socket 传输。在数据的接收与路由识别方面，系统通过 SocketInstance 监听 message 事件，调用 handleMqttMessage 函数，提取控制字节并与 0xF0 进行按位与操作，经由流式解码器解析出主题与负载，最后通过 this.callback(topic, payload) 完成回调分发。

(2) 看门狗机制

看门狗机制的核心在于喂狗操作。每次接收到有效数据时，分别更新 `lastrgbDateTime` 与 `lastsoiDateTime` 为当前时间戳。系统设置 `BOARD_TIMEOUT_MS` 为 5000 毫秒，并通过 `this.boardOfflineCheckTimer` 定时器（间隔 1000 毫秒）定时检查设备在线状态。当超时触发时，将 `isBoardRgbConnected` 标志位置为 `false`，并记录 `LogType.ERROR` 级别日志，实现状态级联响应。

(3) 心跳保护机制

心跳保护机制首先涉及心跳周期的设计。系统将心跳周期设置为保活时间（`keepAliveSeconds`）的 0.75 倍，以容忍网络传输延迟，确保服务器不会因网络抖动而误判客户端离线。在双向握手验证方面，系统维护 `PingOutstanding` 旗标，客户端发送 `0xC0` 心跳请求，服务器收到后立即回复 `0xD0` 响应。若超时未收到响应，触发超时惩罚，执行状态清理后启动异步退避重连，再次完成 TCP 握手。

(4) 报错与自动重连机制

报错与自动重连机制实现三层异常捕获，包括 Socket 链路异常、Socket 关闭通知及 Promise 异常。状态清理阶段调用 `disconnectInternal()` 函数，释放 Socket 连接与心跳定时器。随后启动异步退避重连，通过 `setInterval` 结合 `setTimeout` 方式，每隔 5000 毫秒尝试一次 TCP 握手，直至连接恢复。

4. 状态管理与数据模型

(1) 总览

(2) 控制状态管理

```
@StorageLink("vibrationON") vibrationON: boolean = true;
@StorageLink("isWideScreen") isWideScreen: boolean = true;
@StorageLink("isBoardRgbConnected") isBoardRgbConnected: boolean = false;
@StorageLink("isBoardSoiConnected") isBoardSoiConnected: boolean = false;
@StorageLink("autoEnabled") autoEnabled: boolean = false;

@StorageLink('temp') temp: number = 28;
@StorageLink('hum') hum: number = 50;
@StorageLink('intensity') intensity: number = 134;
@StorageLink('moisture') moisture: number = 50;
@StorageLink('fanValue') fanValue: number = 0;
@StorageLink('waterPumpValue') waterPumpValue: number = 0;
@StorageLink('ledValue') ledValue: number = 0;
@StorageLink('zone1') zone1: ZoneData = new ZoneData();
public mqttClient!: MqttReceiverClient;
```

```
@StorageLink("transparentColor") transparentColor: ResourceColor = $r("sys.color.ohos_toggle_bg_transparent");
@StorageLink("transparentDiffColor") transparentDiffColor: ResourceColor = $r("sys.color.ohos_id_color_mask_thin_transparent");
@StorageLink("transparentBorderColor") transparentBorderColor: ResourceColor = $r("sys.color.ohos_toast_border_color");
@StorageLink("lightColor") lightColor: ResourceColor = Color.White;
@StorageLink("lightIntensity") lightIntensity: number = 10;
@StorageLink("illuminatedType") illuminatedType: hdsEffect.PointLightIlluminatedType = hdsEffect.PointLightIlluminatedType.NONE;
@State button_gradient_state: hdsEffect.PressShadowType = hdsEffect.PressShadowType.NONE;
@State isPressedConnect: boolean = false;
@State isPressedRGBConnect: boolean = false;
@State isPressedSOIConnect: boolean = false;
@State isPressedD: boolean = false;
@State isPressedI: boolean = false;
@State isPressedC: boolean = false;
@State isPressed2: boolean = false;
@State isPressed3: boolean = false;
@State lightSourceTypeConnect: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceTypeRGBConnect: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceTypeSOIConnect: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceTypeA: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceTypeB: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceTypeC: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceTypeD: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceTypeControl: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceType1: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceType2: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State lightSourceType3: hdsEffect.PointLightSourceType = hdsEffect.PointLightSourceType.NONE;
@State controller: hdsEffect.ShaderEffectController = new hdsEffect.ShaderEffectController();
@State animStep1: number = 0;
@State animStep2: number = 0;
```

(3) 参数传递

```
HdsTabs({ controller: this.TabBarController }) {
  TabContent() {
    ManualPage({ mqttClient: this.mqttClient, isVisible: this.currentTabIndex === 0 })
    .tabBar(new BottomTabBarStyle($r('sys.media.wifi_router_fill'), "自定义模式"))

    TabContent() {
      AutoPage({ mqttClient: this.mqttClient, isVisible: this.currentTabIndex === 1 })
    }
    .tabBar(new BottomTabBarStyle($r('sys.media.wifi_router_fill'), "智能模式"))
  }
  .onChange((index: number) => {
    if (this.vibrationON) vibration.dotVibration();
    this.currentTabIndex = index;
  })
}
```

```
private toggleSplitMode(isSplit: boolean) {
  animateTo({
    curve: curves.springMotion(0.4, 0.8),
    duration: 400
  }, () => {
    this.navBarWidth = isSplit ? '40%' : '100%';
  })
}
```

(4) 全局日志同步

```
export class GlobalLogBus {
  private static onLogEmit: ((type: LogType, content: string) => void) | null = null;
  public static register(cb: (type: LogType, content: string) => void) {
    GlobalLogBus.onLogEmit = cb;
  }
  public static unregister() {
    GlobalLogBus.onLogEmit = null;
  }
  public static emit(type: LogType, content: string) {
    if (GlobalLogBus.onLogEmit) {
      GlobalLogBus.onLogEmit(type, content);
    }
  }
}
```

```
private printLog(type: LogType, message: string): void {
  GlobalLogBus.emit(type, 'Manual: ${message}');
  const now = new Date();
  const timeStr = `${now.toLocaleTimeString()}.${String(now.getMilliseconds()).padStart(3, '0')}`;
  const logHeader = `[%{timeStr}] [%{type}]`;
  if (type === LogType.ERROR) {
    console.error(`${logHeader} ${message}`);
  } else {
    console.info(`${logHeader} ${message}`);
  }
}
```

```
@State @Watch('onLogListChange') logList: LogMessage[] = [];
private dataSource: LogDataSource = new LogDataSource();
onLogListChange() {
  this.dataSource.reloadData(this.logList);
  if (this.isAtBottom) {
    setTimeout(() => { this.scroller.scrollToEdge(Edge.Bottom); }, 50);
  }
}
private getTypeColor(type: LogType): Color | string {
  switch (type) {
    case LogType.SEND: return this.autoEnabled? '#ff0055ff' : '#0070ff';
    case LogType.RECEIVE: return this.autoEnabled? '#ff126600' : '#ff22b17';
    case LogType.INFO: return this.autoEnabled? '#444444' : '#666666';
    case LogType.ERROR: return '#ff2222';
    default: return this.autoEnabled? '#111111' : '#333333';
  }
}
private getLogContent(content: string): string {
  if (content.includes('Rayana/rgb/')) || content.includes('RGB开发板') { return '[RGB]' + content; }
  else if (content.includes('Rayana/soi/')) || content.includes('SOI开发板') { return '[SOI]' + content; }
  return content;
}
private getLabel(type: LogType): string {
  switch (type) {
    case LogType.SEND: return 'SENT';
    case LogType.RECEIVE: return 'RECV';
    case LogType.INFO: return 'INFO';
    case LogType.ERROR: return 'ERR';
    default: return 'LOG';
  }
}
private getTime(timeStr: string): string { return timeStr.split(':').map(num => num.padStart(2, '0')).join(':'); }
```

(5) 生命周期管理

```
aboutToAppear(): void {
  GlobalLogBus.register((type: LogType, content: string) => {
    const now = new Date();
    const timeStr = `${now.getHours()}:${now.getMinutes()}:${now.getSeconds()}`;
    this.logList.push({ id: Date.now(), time: timeStr, type, content });
    if (this.logList.length > 100) this.logList.shift();
  });
  try {
    this.mqttClient = new MqttReceiverClient((topic: string, payload: string) => {
      this.onMqttMessage(topic, payload);
    });
    this.mqttClient.setLogCallback((type: LogType, content: string) => {
      GlobalLogBus.emit(type, content);
    });
    this.mqttClient.onStart();
  } catch (error) {
    console.error('App 初始化失败: ${JSON.stringify(error)}');
    this.isBoardRgbConnected = false;
    this.isBoardSoiConnected = false;
    promptAction.showToast({ message: '板端通讯组件初始化失败', duration: 3000 });
  }
  if (this.isWideScreen) { this.toggleSplitMode(true); this.pageStack.pushPathByName('ConsolePage', null); }
  else this.toggleSplitMode(false);
}
aboutToDisappear(): void {
  this.zone1.isExit = true;
  GlobalLogBus.unregister();
  if (this.zone1.autoManagerThread !== null) {
    clearInterval(this.zone1.autoManagerThread);
    this.zone1.autoManagerThread = null;
  }
  if (this.mqttClient) {
    this.mqttClient.disconnect();
  }
}
onBackPressed(): boolean | void {
  if (this.vibrationON) vibration.dotVibration();
  if (this.pageStack.size() > 0) {
    this.pageStack.pop();
    if (this.pageStack.size() === 0) {
      this.toggleSplitMode(false);
    }
    return true;
  }
}
```

```
aboutToAppear(): void {
  this.startEnterAnimation();
  this.dataSource.reloadData(this.logList);
}
aboutToDisappear(): void {
  this.stopEnterAnimation();
}
onPageShow() {
  this.scroller.scrollToEdge(Edge.Bottom);
}
onLogListChange() {
  this.dataSource.reloadData(this.logList);
  if (this.isAtBottom) {
    setTimeout(() => {
      this.scroller.scrollToEdge(Edge.Bottom);
    }, 50);
  }
}
```

```

aboutToAppear(): void {
    this.isWideScreen = wx2vp(display.getDefaultDisplaySync().width) > 600 ? true : false;
    this.autoEnabled = false;
    this.printLog(LogType.INFO, "已切换到自定义模式");

    if (this.isVisible) {
        this.handleManualPageVisibility(true);
    }
}
aboutToDisappear(): void {
    this.handleManualPageVisibility(false);
}

```

5. 智能控制策略

```

private startAutoLogic(): void {
    this.stopAutoLogic();
    this.zone1.isExit = false;
    this.printLog(LogType.INFO, "智能模式启动");
    const handler = setInterval(() => {
        if (this.zone1.isExit) {
            clearInterval(handler);
            return;
        }
        if (!this.autoEnabled) {
            return;
        }
        try {
            const tempMax = parseInt(this.zone1.autoTempMax) || 30;
            const humMax = parseInt(this.zone1.autoHumMax) || 80;
            const moistureMin = parseInt(this.zone1.autoMoistureMin) || 40;
            const moistureMax = parseInt(this.zone1.autoMoistureMax) || 75;
            const intensityMin = parseInt(this.zone1.autoIntensityMin) || 200;
            if (this.isBoardSslConnected) {
                let targetFan = 0;
                if (this.temp > tempMax + 10 || this.hum > humMax + 20 || this.moisture > moistureMax + 20) {
                    targetFan = 3;
                } else if (this.temp > tempMax + 5 || this.hum > humMax + 10 || this.moisture > moistureMax + 10) {
                    targetFan = 2;
                } else if (this.temp > tempMax || this.hum > humMax || this.moisture > moistureMax) {
                    targetFan = 1;
                }
                if (this.fanValue !== targetFan) {
                    this.printLog(LogType.INFO, `[策略变更] 触发降温降湿, 风扇状态跳转: ${this.fanValue} -> ${targetFan} 档`);
                    this.fanValue = targetFan;
                    this.safePublish(TOPIC_S01_FAN, targetFan.toString());
                }
            }
            if (this.isBoardSslConnected) { ... }
            if (this.isBoardRgcConnected) { ... }
        } catch (err) { ... }
    }, 2000);
    this.zone1.autoManagerThread = handler;
}

```

```

private stopAutoLogic(): void {
    this.zone1.isExit = true;
    if (this.zone1.autoManagerThread !== null) {
        this.printLog(LogType.INFO, "已停止并回收智能自动监测线程锁");
        clearInterval(this.zone1.autoManagerThread);
        this.zone1.autoManagerThread = null;
    }
}

```

6. UI 界面设计

(1) 视觉风格

色彩风格：系统视觉基调基于 color.json 与页面自定颜色定义。总体以轻盈的科技灰蓝构建多层次背景，营造通透干净的数字大棚视界。核心状态色提取自自然，采用明暗双层

逻辑：生态绿与轻绿交织，用于设备在线及正常运作提示；特殊页面使用警告红配合轻红，用于异常与断连警示。色彩规范高度统一，符合用户直觉。

(2) 字体与层级

遵循 HarmonyOS 美学规范，字体与页面层级严格划分。数据看板的核心监测数值采用 22px 粗体（如空气温度、土壤湿度），确保核心指标一目了然；模块标题采用 17px 强调体，平衡页面结构；辅助信息与设备标签则采用 12 至 14px 常规体。整体排版字阶错落有致，在保障大屏与手机端跨端自适应的同时，提升了数据可读性。

(3) 图标与插图

系统图标采用 app.media 与 sys.symbol 双系统设计，融合了业务定制与鸿蒙原生美学。设备控制类图标（如风扇、水泵）使用 app.media 引入高辨识度的微拟物线框图标，滑块联动直观；页面组件则深度集成鸿蒙原生的 sys.symbol 图标系统，完美契合系统级视觉语言。响应式界面布局

(4) 动效：引力，光场，转场

7. Smart Shed Glass

Hdsbuttons 组件

(5) 玻璃质感&气态材质 @HdsButtons

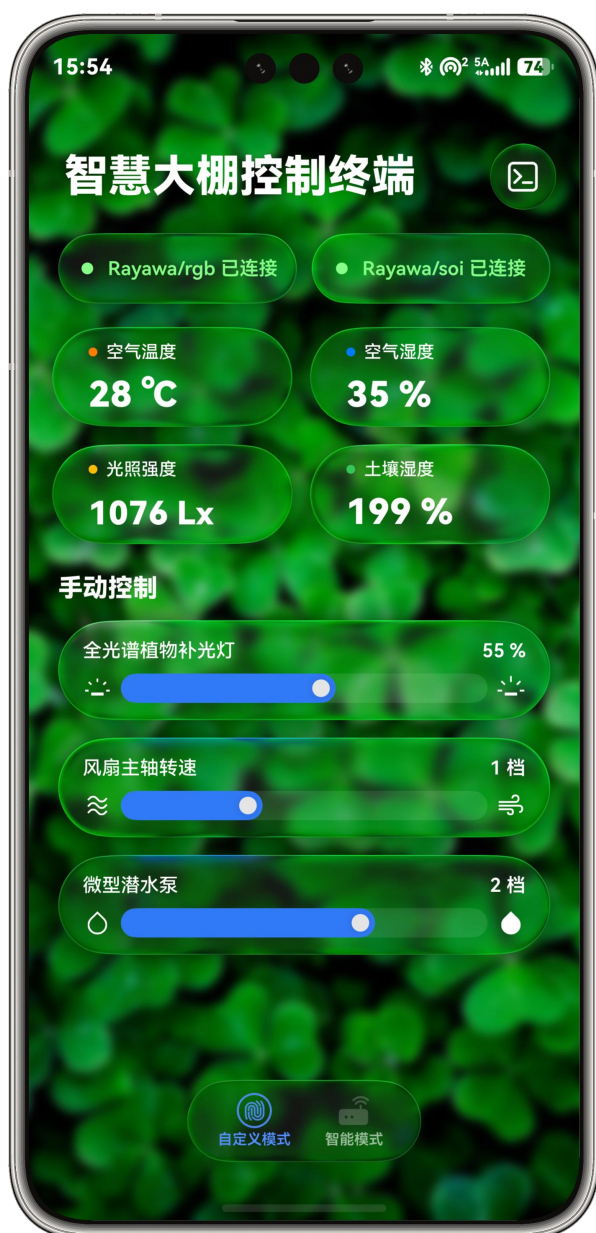
materialType: hdsMaterial.MaterialType.IMMERSIVE

materialLevel: hdsMaterial.MaterialLevel.EXQUISITE

.gesture() -> .scale({ x, y }) & .offset({ x, y })

点光源&按压阴影 @HdsButtons

点光源&按压阴影 @HdsButtons



五、项目创新点总结

(一) 嵌入式端创新

本系统在多线程并发设计的同时，有效避免了扩展板引脚冲突问题。采用双开发板并行设计，确保两者之间通信稳定。文件结构清晰，针对不同板卡进行编译时，仅需切换 GN 注释即可完成配置。该设计在实现极强代码扩展性的同时，保证了程序的轻量化。

(二) 通信架构创新

系统摒弃传统热点直连方式，采用云端中转架构。基于网页实现，用户随时随地可用，调试简便轻量。不依赖特定电脑，无需安装文件，日志打印清晰，通信稳定可靠。

(三) 应用端创新

系统抛弃传统 Java UI，采用面向未来的 ArkUI 鸿蒙设计语言，支持自定义光场效果与沉浸式材质。在日志方面，提供友好的用户提示与界面设计。MQTT 逻辑得到强化，并引入新时代路由栈架构。

致 谢

本项目的顺利完成，离不开多位师长、同学与家人的支持与帮助，在此谨致以诚挚的感谢。

特别感谢数字产业学院姜放放老师对本项目的鼎力支持。姜老师提供了开发板调试环境、SSD 运行虚拟机，以及博远 701 与 704 场地，助力我专注深耕鸿蒙全栈开发与学习；

感谢中软国际教育王岚老师进行为期一学期的三段式授课，并提供了丰富的课程资源与项目内容。王老师在项目制综合实践中的指导，为本项目奠定了重要基础；

感谢所有参与答辩的老师和同学。特别感谢数字产业学院姜放放老师、中软国际教育王岚老师、计算机科学与技术李源老师，以及计算机科学与技术 24 级李焱龙同学、北京信息科技大学软件工程 25 级黄继洲同学特别到场聆听答辩。感谢在场所有《鸿蒙项目制综合实践》课程同学的宝贵意见与支持；

感谢我的父母，以及 Tracy、Joe、Youca 在我学习与项目开发过程中给予的理解与支持；

感谢所有正常工作运行的生产力设备。感谢 OpenAI、Google、深度求索、智谱 AI 等公司提供的大语言模型技术，在代码调试、文档撰写与网页设计过程中提供了有力辅助；

再次感谢所有直接或间接帮助过本项目的老师与同学。

附 录

页面内容	URL
鸿蒙应用端	https://github.com/Rayawa/SmartShed-HarmonyOS
Hi3861 嵌入式端	https://github.com/Rayawa/SmartShed-Hi3861
项目主页	https://rayawa.top/projects/SmartShed.html



智慧大棚 Smart Shed

rayawa.top/projects/SmartShed

- 代码仓库
- 项目论文与演示下载
- 项目简介
- 各部分详细展示





SmartShed-Hi3861

 OpenHarmony
<https://github.com/Rayawa/SmartShed-Hi3861>





SmartShed-HarmonyOS

<https://github.com/Rayawa/SmartShed-HarmonyOS>

